HyperV Agent API - Full Documentation

📋 Overview

HyperV Agent provides a comprehensive REST API for remote Hyper-V management through a centralized management application. The API supports multi-host management scenarios with full CRUD operations for virtual machines, containers, networks, and storage.

Base Information

- **API Version:** 1.0.0
- **Base URL:** `http://{host-ip}:8743/api/v1`
- **Protocol:** HTTP/HTTPS
- **Content-Type:** `application/json`
- **Authentication:** None (host-level security)
- **Documentation:** Swagger UI available at `/swagger`
- **CORS:** ☑ **CONFIGURED** - Supports cross-origin requests for web applications

Architecture

- **HCS Backend:** Container and lightweight VM management
- **WMI Backend:** Full-featured VM management and traditional Hyper-V operations
- **HCN Backend:** Modern networking stack for containers and VMs
- **Storage Services:** VHD management and storage operations

---

📊 Service Management

1. Health Check

Monitor agent availability and service status.

Get Health Status

```
GET /api/v1/service/health
```

Response: 200 OK

```
{
  "status": "healthy",
  "timestamp": "2025-01-11T14:30:00.123Z",
  "version": "1.0.0",
  "services": {
    "hcs": "available",
    "wmi": "available",
```

```
    "hcn": "available"
  }
}
```

Response Fields:

- `status`: Service health status (`healthy` | `degraded` | `unhealthy`)
- `timestamp`: Current UTC timestamp
- `version`: Agent version
- `services`: Backend service availability

2. Agent Information

Get comprehensive agent information and capabilities.

Get Agent Information

```
GET /api/v1/service/info
```

Response: 200 OK

```
{
  "name": "HyperV Agent",
  "version": "1.0.0",
  "description": "HyperV management agent providing REST API for VM, container,
network and storage operations",
  "endpoints": {
    "vms": "/api/v1/vms",
    "containers": "/api/v1/containers",
    "networks": "/api/v1/networks",
    "storage": "/api/v1/storage"
  },
  "capabilities": [
    "VM management (HCS & WMI)",
    "Container management",
    "Network management",
    "Storage management",
    "Snapshot operations",
    "Replication services"
  ]
}
```

🖥 Virtual Machine Management

3. VM Discovery and Listing

List All Virtual Machines

```
GET /api/v1/vms
```

Response: 200 OK

```
{
  "HCS": {
    "Count": 2,
    "VMs": [
      {
        "Name": "container-vm-1",
        "State": "Running",
        "Id": "hcs-vm-001"
      }
    ],
    "Backend": "HCS"
  },
  "WMI": {
    "Count": 3,
    "VMs": [
      {
        "Name": "Production-VM-01",
        "State": "Running",
        "Id": "wmi-vm-001",
        "Memory": 4096,
        "Processors": 2
      }
    ],
    "Backend": "WMI"
  }
}
```

Check VM Presence

```
GET /api/v1/vms/{vmName}/present
```

Path Parameters:

- `vmName`: Name of the virtual machine

Response: 200 OK

```
{
  "present": true,
```

```
  "hcs": false,
  "wmi": true
}
```

4. VM Lifecycle Management

Create Virtual Machine

```
POST /api/v1/vms
```

Request Body:

```
{
  "Id": "vm-550e8400-e29b-41d4-a716-446655440000",
  "Name": "ProductionVM-01",
  "MemoryMB": 4096,
  "CpuCount": 4,
  "DiskSizeGB": 100,
  "Mode": "WMI"
}
```

Request Fields:

- `Id`: Unique VM identifier (required)
- `Name`: VM display name (required)
- `MemoryMB`: Memory allocation in MB (required)
- `CpuCount`: Number of virtual CPUs (required)
- `DiskSizeGB`: Primary disk size in GB (required)
- `Mode`: Backend mode (`HCS` | `WMI`)

Response: 200 OK

```
{
  "Id": "vm-550e8400-e29b-41d4-a716-446655440000",
  "Name": "ProductionVM-01",
  "Status": "Created",
  "Backend": "WMI",
  "CreatedAt": "2025-01-11T14:30:00Z"
}
```

Error Responses:

- **400 Bad Request:** Invalid request data
- **500 Internal Server Error:** Backend service error

Start Virtual Machine

```
POST /api/v1/vms/{vmName}/start
```

Response: 200 OK (Empty body)

Error Responses:

- **404 Not Found:** VM not found
- **500 Internal Server Error:** Failed to start VM

Stop Virtual Machine

```
POST /api/v1/vms/{vmName}/stop
```

Response: 200 OK (Empty body)

Graceful Shutdown

```
POST /api/v1/vms/{vmName}/shutdown
```

Response: 200 OK (Empty body)

Force Terminate

```
POST /api/v1/vms/{vmName}/terminate
```

Response: 200 OK (Empty body)

Pause Virtual Machine

```
POST /api/v1/vms/{vmName}/pause
```

Response: 200 OK (Empty body)

Resume Virtual Machine

```
POST /api/v1/vms/{vmName}/resume
```

Response: 200 OK (Empty body)

Save VM State

```
POST /api/v1/vms/{vmName}/save
```

Response: 200 OK (Empty body)
Note: Only supported for WMI VMs. HCS VMs return 501 Not Implemented.

5. VM Configuration

Get VM Properties

```
GET /api/v1/vms/{vmName}/properties
```

Response: 200 OK

```
{
  "backend": "WMI",
  "properties": {
    "Name": "ProductionVM-01",
    "State": "Running",
    "Memory": 4096,
    "Processors": 2,
    "Generation": 2,
    "Version": "10.0",
    "IntegrationServicesState": "Up to date",
    "Heartbeat": "OK",
    "Uptime": "2.15:30:45"
  }
}
```

Configure VM Settings

```
POST /api/v1/vms/{vmName}/configure
```

Request Body:

```
{
  "StartupMemoryMB": 8192,
  "CpuCount": 4,
```

```
    "Notes": "Production database server",
    "EnableDynamicMemory": true,
    "MinimumMemoryMB": 1024,
    "MaximumMemoryMB": 16384,
    "TargetMemoryBuffer": 20,
    "VirtualMachineReserve": 0,
    "VirtualMachineLimit": 0,
    "RelativeWeight": 100,
    "LimitProcessorFeatures": false,
    "MaxProcessorsPerNumaNode": 12,
    "MaxNumaNodesPerSocket": 1,
    "HwThreadsPerCore": 0
  }
```

Response: 200 OK (Empty body)

Error Responses:

- **404 Not Found:** VM not found
- **501 Not Implemented:** Configuration not supported for HCS VMs

Modify VM Configuration

```
POST /api/v1/vms/{vmName}/modify
```

Request Body: JSON configuration string
Response: 200 OK (Empty body)

6. VM Snapshot Management

List VM Snapshots

```
GET /api/v1/vms/{vmName}/snapshots
```

Response: 200 OK

```
[
  {
    "Id": "snapshot-001",
    "Name": "Before Windows Update",
    "CreationTime": "2025-01-11T10:00:00Z",
    "Type": "Standard",
    "ParentSnapshot": null
  },
  {
```

```
    "Id": "snapshot-002",
    "Name": "After Configuration",
    "CreationTime": "2025-01-11T14:00:00Z",
    "Type": "Standard",
    "ParentSnapshot": "snapshot-001"
  }
]
```

Create VM Snapshot

```
POST /api/v1/vms/{vmName}/snapshots
```

Request Body:

```
{
  "SnapshotName": "Pre-deployment checkpoint",
  "Notes": "Snapshot created before application deployment"
}
```

Response: 200 OK

```
{
  "snapshotId": "snapshot-003",
  "name": "Pre-deployment checkpoint",
  "createdAt": "2025-01-11T15:00:00Z"
}
```

Delete VM Snapshot

```
DELETE /api/v1/vms/{vmName}/snapshots/{snapshotId}
```

Response: 200 OK (Empty body)

Revert VM to Snapshot

```
POST /api/v1/vms/{vmName}/snapshots/{snapshotId}/revert
```

Response: 200 OK (Empty body)

7. VM Storage Management

Get VM Storage Devices

```
GET /api/v1/vms/{vmName}/storage/devices
```

Response: 200 OK

```
[
  {
    "DeviceId": "device-001",
    "Name": "Primary Hard Drive",
    "DeviceType": "VirtualDisk",
    "Path": "C:\\VMs\\ProductionVM-01\\disk1.vhdx",
    "ControllerId": "scsi-controller-0",
    "ControllerType": "SCSI",
    "IsReadOnly": false,
    "OperationalStatus": "Connected",
    "Location": 0
  }
]
```

Add Storage Device to VM

```
POST /api/v1/vms/{vmName}/storage/devices
```

Request Body:

```
{
  "DeviceType": "VirtualDisk",
  "Path": "C:\\VMs\\ProductionVM-01\\data.vhdx",
  "ReadOnly": false,
  "ControllerId": "scsi-controller-0"
}
```

Response: 200 OK

```
{
  "message": "Storage device added successfully",
  "vmName": "ProductionVM-01"
}
```

Remove Storage Device from VM

```
DELETE /api/v1/vms/{vmName}/storage/devices/{deviceId}
```

Response: 200 OK

```
{
  "message": "Storage device removed successfully",
  "vmName": "ProductionVM-01",
  "deviceId": "device-001"
}
```

Get VM Storage Controllers

```
GET /api/v1/vms/{vmName}/storage/controllers
```

Response: 200 OK

```
[
  {
    "ControllerId": "scsi-controller-0",
    "Name": "SCSI Controller",
    "ControllerType": "SCSI",
    "MaxDevices": 64,
    "AttachedDevices": 2,
    "SupportsHotPlug": true,
    "OperationalStatus": "OK",
    "Protocol": "SCSI",
    "AvailableLocations": [2, 3, 4, 5]
  }
]
```

📦 Container Management

8. Container Lifecycle

Create Container

```
POST /api/v1/containers
```

Request Body:

```
{
  "Id": "container-550e8400-e29b-41d4-a716-446655440000",
  "Name": "WebServer-Container",
  "Image": "mcr.microsoft.com/windows/servercore:ltsc2022",
  "MemoryMB": 2048,
  "CpuCount": 2,
  "StorageSizeGB": 20,
  "Mode": "HCS",
  "Environment": {
    "ASPNETCORE_ENVIRONMENT": "Production",
    "DATABASE_URL": "server=db-host;database=app"
  },
  "PortMappings": {
    "80": "8080",
    "443": "8443"
  },
  "VolumeMounts": {
    "C:\\app\\logs": "C:\\host\\logs",
    "C:\\app\\data": "C:\\host\\data"
  }
}
```

Request Fields:

- `Id`: Unique container identifier
- `Name`: Container display name
- `Image`: Container image URI
- `MemoryMB`: Memory limit in MB
- `CpuCount`: CPU core count
- `StorageSizeGB`: Storage allocation in GB
- `Mode`: Backend mode (`HCS` | `WMI`)
- `Environment`: Environment variables (key-value pairs)
- `PortMappings`: Port mappings (container:host)
- `VolumeMounts`: Volume mount points (container:host)

Response: 200 OK

```
{
  "Id": "container-550e8400-e29b-41d4-a716-446655440000",
  "Name": "WebServer-Container",
  "Status": "Created",
  "Mode": "HCS"
}
```

List All Containers

```
GET /api/v1/containers
```

Response: 200 OK

```
{
  "HcsContainers": [
    {
      "Id": "container-001",
      "Name": "WebServer-Container",
      "State": "Running",
      "Image": "mcr.microsoft.com/windows/servercore:ltsc2022"
    }
  ],
  "WmiContainers": [],
  "Message": "Container listing reflects current runtime state"
}
```

Get Container Information

```
GET /api/v1/containers/{containerId}
```

Response: 200 OK

```
{
  "backend": "HCS",
  "properties": {
    "Id": "container-001",
    "Name": "WebServer-Container",
    "State": "Running",
    "Image": "mcr.microsoft.com/windows/servercore:ltsc2022",
    "Memory": 2048,
    "CPUs": 2,
    "NetworkMode": "NAT",
    "CreatedAt": "2025-01-11T10:00:00Z",
    "StartedAt": "2025-01-11T10:00:15Z"
  }
}
```

Container State Management

```
POST /api/v1/containers/{containerId}/start      # Start container
POST /api/v1/containers/{containerId}/stop       # Stop container
```

```
POST /api/v1/containers/{containerId}/terminate  # Force terminate
POST /api/v1/containers/{containerId}/pause      # Pause container
POST /api/v1/containers/{containerId}/resume     # Resume container
```

Response: 200 OK

```
{
  "message": "HCS container {containerId} started successfully"
}
```

Delete Container

```
DELETE /api/v1/containers/{containerId}
```

Response: 200 OK

```
{
  "message": "Container container-001 deleted successfully"
}
```

🌐 Network Management (HCN)

9. Network Operations

Create NAT Network

```
POST /api/v1/networks/nat?name={networkName}&prefix={cidr}
```

Query Parameters:

- `name`: Network name (required)
- `prefix`: CIDR subnet (default: `192.168.100.0/24`)

Response: 200 OK

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000"
}
```

Delete Network

```
DELETE /api/v1/networks/{networkId}
```

Path Parameters:

- `networkId`: Network GUID

Response: 200 OK (Empty body)
Error Response: 404 Not Found (Network not found)

Get Network Properties

```
GET /api/v1/networks/{networkId}/properties?query={query}
```

Query Parameters:

- `query`: Property query filter (optional)

Response: 200 OK

```
{
  "Id": "550e8400-e29b-41d4-a716-446655440000",
  "Name": "Production-NAT",
  "Type": "NAT",
  "Subnet": "192.168.100.0/24",
  "Gateway": "192.168.100.1",
  "DNSServers": ["8.8.8.8", "8.8.4.4"],
  "State": "Active",
  "Policies": []
}
```

10. Network Endpoint Management

Create Network Endpoint

```
POST /api/v1/networks/{networkId}/endpoints?name={endpointName}&ipAddress={ip}
```

Query Parameters:

- `name`: Endpoint name (required)
- `ipAddress`: IP address (optional, auto-assigned if empty)

Response: 200 OK

```
{
  "id": "endpoint-550e8400-e29b-41d4-a716-446655440001"
}
```

Delete Network Endpoint

```
DELETE /api/v1/networks/endpoints/{endpointId}
```

Response: 200 OK (Empty body)

Get Endpoint Properties

```
GET /api/v1/networks/endpoints/{endpointId}/properties?query={query}
```

Response: 200 OK

```
{
  "Id": "endpoint-550e8400-e29b-41d4-a716-446655440001",
  "Name": "VM-Endpoint-01",
  "IPAddress": "192.168.100.10",
  "MacAddress": "00-15-5D-00-00-01",
  "NetworkId": "550e8400-e29b-41d4-a716-446655440000",
  "State": "Connected"
}
```

💾 Storage Management

11. VHD Operations

Create Virtual Hard Disk

```
POST /api/storage/vhd
```

Request Body:

```
{
  "Path": "C:\\VMs\\ProductionVM-01\\data.vhdx",
  "MaxInternalSize": 107374182400,
  "Format": "VHDX",
  "Type": "Dynamic"
}
```

Request Fields:

- `Path`: VHD file path (required)
- `MaxInternalSize`: Maximum size in bytes (required)
- `Format`: VHD format (`VHD` | `VHDX`)
- `Type`: VHD type (`Fixed` | `Dynamic` | `Differencing`)

Response: 200 OK

```
{
  "message": "VHD created successfully",
  "path": "C:\\VMs\\ProductionVM-01\\data.vhdx"
}
```

Attach VHD to VM

```
PUT /api/storage/vhd/attach?vmName={vmName}&vhdPath={path}
```

Response: 200 OK

```
{
  "message": "VHD attached successfully",
  "vmName": "ProductionVM-01",
  "vhdPath": "C:\\VMs\\ProductionVM-01\\data.vhdx"
}
```

Detach VHD from VM

```
PUT /api/storage/vhd/detach?vmName={vmName}&vhdPath={path}
```

Response: 200 OK

```
{
  "message": "VHD detached successfully",
  "vmName": "ProductionVM-01",
  "vhdPath": "C:\\VMs\\ProductionVM-01\\data.vhdx"
}
```

Resize Virtual Hard Disk

```
PUT /api/storage/vhd/resize
```

Request Body:

```
{
  "Path": "C:\\VMs\\ProductionVM-01\\data.vhdx",
  "MaxInternalSize": 214748364800
}
```

Response: 200 OK

```
{
  "message": "VHD resized successfully",
  "path": "C:\\VMs\\ProductionVM-01\\data.vhdx",
  "newSize": 214748364800
}
```

12. Advanced VHD Operations

Get VHD Metadata

```
GET /api/storage/vhd/metadata?vhdPath={path}
```

Response: 200 OK

```
{
  "Path": "C:\\VMs\\ProductionVM-01\\data.vhdx",
  "Format": "VHDX",
  "Type": "Dynamic",
  "UniqueId": "550e8400-e29b-41d4-a716-446655440000",
  "VirtualSize": 214748364800,
  "PhysicalSize": 1073741824,
```

```
  "IsAttached": true,
  "AttachedTo": "ProductionVM-01",
  "BlockSize": 33554432,
  "LogicalSectorSize": 512,
  "PhysicalSectorSize": 4096
}
```

Update VHD Metadata

```
PUT /api/storage/vhd/metadata?vhdPath={path}
```

Request Body:

```
{
  "ItemName": "Production Data Disk",
  "Description": "Primary data storage for production database",
  "Source": "Template-Database-Disk.vhdx",
  "SourceDisk": "C:\\Templates\\Database.vhdx"
}
```

Compact VHD

```
POST /api/storage/vhd/compact
```

Request Body:

```
{
  "Path": "C:\\VMs\\ProductionVM-01\\data.vhdx",
  "Mode": "Quick"
}
```

Response: 202 Accepted

```
{
  "message": "VHD compaction started successfully",
  "path": "C:\\VMs\\ProductionVM-01\\data.vhdx",
  "jobId": "compact-job-001"
}
```

Convert VHD Format

```
POST /api/storage/vhd/convert
```

Request Body:

```
{
  "SourcePath": "C:\\VMs\\old-vm.vhd",
  "DestinationPath": "C:\\VMs\\new-vm.vhdx",
  "TargetFormat": "VHDX"
}
```

Response: 202 Accepted

```
{
  "message": "VHD conversion started successfully",
  "source": "C:\\VMs\\old-vm.vhd",
  "destination": "C:\\VMs\\new-vm.vhdx",
  "jobId": "convert-job-001"
}
```

13. VHD Set Operations

Convert to VHD Set

```
POST /api/storage/vhdset/convert
```

Request Body:

```
{
  "SourcePath": "C:\\VMs\\vm.vhdx",
  "DestinationPath": "C:\\VMs\\vm.vhdset"
}
```

Get VHD Set Information

```
GET /api/storage/vhdset/{vhdSetPath}/info
```

Response: 200 OK

```
{
  "Path": "C:\\VMs\\vm.vhdset",
  "VirtualSize": 107374182400,
  "PhysicalSize": 5368709120,
  "BlockSize": 33554432,
  "SnapshotCount": 3,
  "ActiveSnapshotId": "snapshot-003"
}
```

Create VHD Set Snapshot

```
POST /api/storage/vhdset/{vhdSetPath}/snapshots
```

Request Body:

```
{
  "Name": "Daily Backup Snapshot",
  "Description": "Automated daily backup checkpoint"
}
```

14. Change Tracking

Enable Change Tracking

```
POST /api/storage/vhd/{vhdPath}/tracking/enable
```

Response: 200 OK

```
{
  "message": "Change tracking enabled successfully",
  "path": "C:\\VMs\\vm.vhdx",
  "trackingId": "tracking-550e8400"
}
```

Get Virtual Disk Changes

```
GET /api/storage/vhd/{vhdPath}/changes?changeTrackingId={trackingId}
```

Response: 200 OK

```
{
  "ChangeTrackingId": "tracking-550e8400",
  "Changes": [
    {
      "ByteOffset": 1048576,
      "ByteLength": 4096,
      "ChangeType": "Modified"
    }
  ],
  "TotalChangedBytes": 4096
}
```

Job Management

15. Storage Job Monitoring

Get All Storage Jobs

```
GET /api/v1/jobs/storage
```

Response: 200 OK

```
[
  {
    "JobId": "job-550e8400-e29b-41d4-a716-446655440000",
    "OperationType": "CompactVHD",
    "State": "Running",
    "StartTime": "2025-01-11T15:00:00Z",
    "EndTime": null,
    "PercentComplete": 45,
    "Description": "Compacting production data disk",
    "TargetPath": "C:\\VMs\\ProductionVM-01\\data.vhdx"
  },
  {
    "JobId": "job-550e8400-e29b-41d4-a716-446655440001",
    "OperationType": "ConvertVHD",
    "State": "Completed",
    "StartTime": "2025-01-11T14:30:00Z",
    "EndTime": "2025-01-11T14:35:00Z",
    "PercentComplete": 100,
    "Description": "VHD format conversion completed"
```

```
    }
  ]
```

Job States:

- `Queued`: Job waiting to start
- `Running`: Job in progress
- `Completed`: Job finished successfully
- `Failed`: Job failed with error
- `Cancelled`: Job was cancelled

Get Storage Job Details

```
GET /api/v1/jobs/storage/{jobId}
```

Response: 200 OK

```
{
  "JobId": "job-550e8400-e29b-41d4-a716-446655440000",
  "OperationType": "CompactVHD",
  "State": "Running",
  "StartTime": "2025-01-11T15:00:00Z",
  "EndTime": null,
  "PercentComplete": 45,
  "Description": "Compacting production data disk",
  "TargetPath": "C:\\VMs\\ProductionVM-01\\data.vhdx",
  "EstimatedTimeRemaining": "00:05:30",
  "BytesProcessed": 4831838208,
  "TotalBytes": 10737418240
}
```

Get Job Affected Elements

```
GET /api/v1/jobs/storage/{jobId}/affected-elements
```

Response: 200 OK

```
[
  {
    "ElementId": "element-550e8400",
    "ElementName": "data.vhdx",
    "ElementType": "VirtualHardDisk",
    "Effects": [
```

```
      {
        "Type": "Modified",
        "Description": "Disk compaction in progress"
      }
    ]
  }
]
```

Cancel Storage Job

```
POST /api/v1/jobs/storage/{jobId}/cancel
```

Response: 200 OK

```
{
  "message": "Storage job cancelled successfully",
  "jobId": "job-550e8400-e29b-41d4-a716-446655440000"
}
```

Delete Completed Job

```
DELETE /api/v1/jobs/storage/{jobId}
```

Response: 200 OK

```
{
  "message": "Storage job deleted successfully",
  "jobId": "job-550e8400-e29b-41d4-a716-446655440000"
}
```

🔧 Advanced Storage Operations

16. Differencing Disks

Create Differencing Disk

```
POST /api/storage/vhd/differencing
```

Request Body:

```
{
  "ChildPath": "C:\\VMs\\VM-Clone\\disk.vhdx",
  "ParentPath": "C:\\Templates\\base-template.vhdx"
}
```

Merge Differencing Disk

```
POST /api/storage/vhd/merge
```

Request Body:

```
{
  "ChildPath": "C:\\VMs\\VM-Clone\\disk.vhdx",
  "DestinationPath": "C:\\VMs\\VM-Merged\\disk.vhdx",
  "MergeDepth": 1
}
```

17. Storage Extent Management

Get Storage Extents

```
GET /api/storage/extents
```

Response: 200 OK

```
[
  {
    "ExtentId": "extent-001",
    "Name": "Physical Extent 1",
    "Type": "Physical",
    "StartingAddress": 0,
    "BlockSize": 512,
    "NumberOfBlocks": 1953525168
  }
]
```

Get Extent Dependencies

```
GET /api/storage/extents/{extentId}/dependencies
```

Get Controller-Device Relationships

```
GET /api/storage/controllers/{controllerId}/devices
GET /api/storage/devices/{deviceId}/controller
GET /api/storage/controllers/{controllerId}/devices/{deviceId}/relationship
```

📊 Data Models

VM Creation Request

```
interface CreateVmRequest {
  Id: string;                    // Unique identifier
  Name: string;                  // VM display name
  MemoryMB: number;              // Memory in MB
  CpuCount: number;              // CPU count
  DiskSizeGB: number;            // Disk size in GB
  Mode: "HCS" | "WMI";           // Backend selection
}
```

Container Creation Request

```
interface CreateContainerRequest {
  Id: string;                           // Unique identifier
  Name: string;                         // Container name
  Image: string;                        // Container image
  MemoryMB: number;                     // Memory limit
  CpuCount: number;                     // CPU count
  StorageSizeGB: number;                // Storage size
  Mode: "HCS" | "WMI";                  // Backend mode
  Environment: Record<string, string>;  // Environment variables
  PortMappings: Record<number, number>; // Port mappings
  VolumeMounts: Record<string, string>; // Volume mounts
}
```

Storage Job Response

```
interface StorageJobResponse {
  JobId: string;                 // Unique job identifier
  OperationType: string;         // Operation type
```

```
  State: JobState;                // Current state
  StartTime: string;              // ISO 8601 timestamp
  EndTime?: string;               // ISO 8601 timestamp
  PercentComplete: number;        // 0-100
  Description: string;            // Job description
  TargetPath?: string;            // Target file path
  EstimatedTimeRemaining?: string; // HH:MM:SS format
  BytesProcessed?: number;        // Bytes processed
  TotalBytes?: number;            // Total bytes
}

type JobState = "Queued" | "Running" | "Completed" | "Failed" | "Cancelled";
```

Storage Device Response

```
interface StorageDeviceResponse {
  DeviceId: string;               // Device identifier
  Name: string;                   // Device name
  DeviceType: string;             // Device type
  Path: string;                   // File path
  ControllerId: string;           // Controller ID
  ControllerType: string;         // Controller type
  IsReadOnly: boolean;            // Read-only flag
  OperationalStatus: string;      // Status
  Location?: number;              // Controller location
}
```

VM Configuration Request

```
interface VmConfigurationRequest {
  StartupMemoryMB?: number;       // Startup memory
  CpuCount?: number;              // CPU count
  Notes?: string;                 // VM notes
  EnableDynamicMemory?: boolean; // Dynamic memory
  MinimumMemoryMB?: number;       // Min memory
  MaximumMemoryMB?: number;       // Max memory
  TargetMemoryBuffer?: number;    // Memory buffer %
  VirtualMachineReserve?: number; // VM reserve %
  VirtualMachineLimit?: number;    // VM limit %
  RelativeWeight?: number;         // CPU weight
  LimitProcessorFeatures?: boolean; // Limit features
  MaxProcessorsPerNumaNode?: number; // NUMA config
  MaxNumaNodesPerSocket?: number;    // Socket config
  HwThreadsPerCore?: number;         // Thread config
}
```

🚨 Error Handling

HTTP Status Codes

Code	Description	Usage
200	OK	Successful operation
202	Accepted	Async operation started
400	Bad Request	Invalid request data
404	Not Found	Resource not found
500	Internal Server Error	Backend service error
501	Not Implemented	Feature not supported by backend

Error Response Format

```
{
  "error": "Detailed error message",
  "details": "Stack trace and technical details",
  "backend": "HCS|WMI|HCN",
  "timestamp": "2025-01-11T15:00:00Z",
  "path": "/api/v1/vms/TestVM/start"
}
```

Common Error Scenarios

VM Not Found

```
{
  "error": "VM 'NonExistentVM' not found",
  "backend": "WMI"
}
```

Feature Not Supported

```
{
  "error": "Snapshots not supported for HCS VMs",
  "backend": "HCS"
}
```

Backend Service Error

```
{
  "error": "Failed to start VM: Access denied",
  "details": "System.UnauthorizedAccessException: Access to the path is
denied",
  "backend": "WMI"
}
```

🔐 Security Considerations

Host-Level Security

- **Elevated Privileges:** Agent requires Administrator privileges
- **Network Access:** Configurable binding (default: localhost only)
- **Authentication:** None (rely on host security)
- **Authorization:** Full access to all operations

Production Deployment

- **Firewall:** Configure appropriate firewall rules
- **Network Isolation:** Use private management networks
- **Monitoring:** Implement audit logging
- **Backup:** Regular configuration backups

📊 Monitoring and Performance

Health Monitoring

```
// Continuous health monitoring
setInterval(async () => {
  try {
    const response = await fetch(`http://${host}:5000/api/v1/service/health`);
    const health = await response.json();
    updateHostStatus(host, health);
  } catch (error) {
    markHostOffline(host);
  }
}, 30000); // Check every 30 seconds
```

Job Progress Tracking

```
// Monitor long-running operations
async function trackStorageJob(hostId, jobId) {
  const response = await
fetch(`http://${hostId}:5000/api/v1/jobs/storage/${jobId}`);
```

```
  const job = await response.json();

  return {
    progress: job.PercentComplete,
    status: job.State,
    eta: job.EstimatedTimeRemaining
  };
}
```

Resource Utilization

```
// Get VM resource usage
async function getVMResources(hostId, vmName) {
  const response = await
fetch(`http://${hostId}:5000/api/v1/vms/${vmName}/properties`);
  const props = await response.json();

  return {
    memory: props.properties.Memory,
    cpus: props.properties.Processors,
    state: props.properties.State
  };
}
```

🏗 Central Management Application Integration

Multi-Host Management

```
class HyperVCentralManager {
  constructor() {
    this.hosts = new Map();
    this.pollingInterval = 30000;
  }

  async addHost(hostId, ipAddress, port = 5000) {
    const baseUrl = `http://${ipAddress}:${port}/api/v1`;

    // Verify connectivity
    const health = await fetch(`${baseUrl}/service/health`);
    if (health.ok) {
      this.hosts.set(hostId, {
        baseUrl,
        status: 'online',
        lastSeen: new Date()
      });
    }
  }
```

```
  async getAllVMs() {
    const allVMs = [];

    for (const [hostId, host] of this.hosts) {
      try {
        const response = await fetch(`${host.baseUrl}/vms`);
        const vms = await response.json();

        allVMs.push({
          hostId,
          hcsVMs: vms.HCS.VMs,
          wmiVMs: vms.WMI.VMs
        });
      } catch (error) {
        console.error(`Failed to get VMs from ${hostId}:`, error);
      }
    }

    return allVMs;
  }

  async startVM(hostId, vmName) {
    const host = this.hosts.get(hostId);
    if (!host) throw new Error(`Host ${hostId} not found`);

    const response = await fetch(`${host.baseUrl}/vms/${vmName}/start`, {
      method: 'POST'
    });

    if (!response.ok) {
      throw new Error(`Failed to start VM: ${response.statusText}`);
    }
  }
}
```

Bulk Operations

```
// Start multiple VMs across hosts
async function startVMsInParallel(vmList) {
  const operations = vmList.map(({hostId, vmName}) =>
    manager.startVM(hostId, vmName)
  );

  const results = await Promise.allSettled(operations);
  return results.map((result, index) => ({
    vm: vmList[index],
    success: result.status === 'fulfilled',
    error: result.reason
```

```
    }));
  }
```

Real-time Updates

```
// WebSocket-like polling for real-time updates
class RealTimeMonitor {
  constructor(manager) {
    this.manager = manager;
    this.subscribers = new Set();
  }

  subscribe(callback) {
    this.subscribers.add(callback);
  }

  async startMonitoring() {
    setInterval(async () => {
      const updates = await this.collectUpdates();
      this.subscribers.forEach(callback => callback(updates));
    }, 5000);
  }

  async collectUpdates() {
    const updates = {
      timestamp: new Date().toISOString(),
      hosts: [],
      vms: [],
      jobs: []
    };

    // Collect data from all hosts
    for (const [hostId, host] of this.manager.hosts) {
      try {
        const [health, vms, jobs] = await Promise.all([
          fetch(`${host.baseUrl}/service/health`).then(r => r.json()),
          fetch(`${host.baseUrl}/vms`).then(r => r.json()),
          fetch(`${host.baseUrl}/jobs/storage`).then(r => r.json())
        ]);

        updates.hosts.push({hostId, health});
        updates.vms.push({hostId, vms});
        updates.jobs.push({hostId, jobs});
      } catch (error) {
        updates.hosts.push({hostId, error: error.message});
      }
    }

    return updates;
```

```
    }
  }
```

📋 Quick Reference

Essential Endpoints for Central Management

Host Discovery

```
GET /service/health          # Check host availability
GET /service/info            # Get host capabilities
```

VM Management

```
GET /vms                     # List all VMs
POST /vms                    # Create VM
GET /vms/{name}/present      # Check VM existence
POST /vms/{name}/start       # Start VM
POST /vms/{name}/stop        # Stop VM
GET /vms/{name}/properties   # Get VM details
```

Container Management

```
GET /containers              # List containers
POST /containers             # Create container
POST /containers/{id}/start  # Start container
POST /containers/{id}/stop   # Stop container
```

Network Management

```
POST /networks/nat           # Create network
GET /networks/{id}/properties # Get network info
POST /networks/{id}/endpoints # Create endpoint
```

Storage Management

```
POST /storage/vhd            # Create VHD
PUT /storage/vhd/attach      # Attach VHD
```

```
GET /jobs/storage           # Monitor jobs
```

🎯 Implementation Recommendations

1. **Host Pool Management**

- Implement host discovery and registration
- Monitor host health and availability
- Handle host failures gracefully
- Load balance operations across hosts

2. **Resource Aggregation**

- Aggregate VM/Container data across hosts
- Provide unified resource views
- Calculate cluster-wide statistics
- Track resource utilization trends

3. **Job Orchestration**

- Queue operations across multiple hosts
- Monitor job progress centrally
- Implement retry mechanisms
- Provide operation history

4. **High Availability**

- Detect host failures
- Implement failover strategies
- Backup configuration data
- Monitor critical services

This comprehensive API documentation provides all the necessary information for building a robust central Hyper-V management application with support for multi-host environments, real-time monitoring, and advanced operations management.

🔗 CORS Support for Web Applications

Problem Solved ☑

HyperV Agent now includes full CORS (Cross-Origin Resource Sharing) support to enable web applications to connect from different origins.

CORS Configuration

The API is configured with two CORS policies in `Program.cs`:

1. Default Policy (Development)

- **Allowed Origins:** `*` (Any origin)
- **Allowed Methods:** `*` (All HTTP methods)
- **Allowed Headers:** `*` (All headers)
- **Credentials:** Not allowed

2. Production Policy

- **Allowed Origins:**
  - `http://localhost:3000` (React dev server)
  - `http://localhost:8080` (Vue dev server)
  - `http://localhost:4200` (Angular dev server)
  - `http://localhost:8000` (Python dev server)
  - `http://management.hyperv.local` (Production HTTP)
  - `https://management.hyperv.local` (Production HTTPS)
- **Allowed Methods:** All HTTP methods
- **Allowed Headers:** All headers
- **Credentials:** Allowed

Usage Example

```
// Now works without CORS errors
fetch('http://127.0.0.1:8743/api/v1/service/info')
  .then(response => response.json())
  .then(data => {
    console.log('Agent Info:', data);
  })
  .catch(error => {
    console.error('Error:', error);
  });
```

Configuration in Central Management App

```
// For production environments, configure API base URL
const API_CONFIG = {
  baseUrl: 'http://127.0.0.1:8743/api/v1',
  timeout: 30000,
  retries: 3
};

class HyperVApiClient {
  constructor(config = API_CONFIG) {
    this.baseUrl = config.baseUrl;
    this.timeout = config.timeout;
  }
```

```
  async makeRequest(endpoint, options = {}) {
    const url = `${this.baseUrl}${endpoint}`;
    const controller = new AbortController();
    const timeoutId = setTimeout(() => controller.abort(), this.timeout);

    try {
      const response = await fetch(url, {
        ...options,
        signal: controller.signal,
        headers: {
          'Content-Type': 'application/json',
          ...options.headers
        }
      });

      clearTimeout(timeoutId);

      if (!response.ok) {
        throw new Error(`HTTP ${response.status}: ${response.statusText}`);
      }

      return response.json();
    } catch (error) {
      clearTimeout(timeoutId);
      throw error;
    }
  }

  // Service endpoints
  async getHealth() {
    return this.makeRequest('/service/health');
  }

  async getInfo() {
    return this.makeRequest('/service/info');
  }

  // VM operations
  async listVMs() {
    return this.makeRequest('/vms');
  }

  async startVM(vmName) {
    return this.makeRequest(`/vms/${vmName}/start`, { method: 'POST' });
  }

  async stopVM(vmName) {
    return this.makeRequest(`/vms/${vmName}/stop`, { method: 'POST' });
  }
}

// Usage in management application
const client = new HyperVApiClient();
```

```
// Test connection
client.getHealth()
  .then(health => console.log('Agent is healthy:', health))
  .catch(error => console.error('Connection failed:', error));
```

Important Notes

- **Port:** Agent runs on port `8743` (not 5000)
- **Binding:** `127.0.0.1:8743` (localhost only for security)
- **CORS:** Enabled by default for development
- **Production:** Configure specific allowed origins
- **Security:** CORS does not replace proper authentication in production

This comprehensive API documentation with CORS support provides all the necessary information for building a robust central Hyper-V management application with full web compatibility, multi-host environments, real-time monitoring, and advanced operations management.